



Vue Router 강의

SPA 라우팅의 핵심부터 가드까지 한 번에

라우트 매핑, 네비게이션, 동적 파라미터, 가드 실습으로 실전 감각을 익히는 완벽 가이드



전체 강의 구성

Vue Router 마스터를 위한 4단계 로드맵

01

싱글 페이지 앱(SPA) 라우터 구조 이해

Routes, RouterView, RouterLink의 역할과 동작 원리

02

라우트 등록 및 네비게이션 실습

RouterLink와 router.push()를 활용한 페이지 이동 구현

03

동적 라우팅과 파라미터 등록

URL 파라미터(:id) 처리와 useRoute() 혹 활용법

네비게이션 가드로 접근 제어하기

인증(Authentication) 미들웨어 구현 및 리다이렉션 처리

최종 결과물



기본 라우팅 프로젝트 완성

SPA 구조가 잡힌 Vue 3 애플리케이션



실전 패턴 적용

프로그래밍 방식 네비게이션 & 동적 파라미터 핸들링



보안 로직 구현

전역 가드를 활용한 로그인 접근 제어 시스템



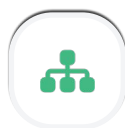
모든 실습은 Vue 3 Composition API 기준으로
진행됩니다.

1

CHAPTER

싱글 페이지 앱(SPA) 라우터 구조 이해하기

SPA에서 주소만 바뀌고 화면을 컴포넌트로 교체하는 라우팅의 핵심 개념과 구조를 살펴봅니다.



ROUTES



VIEW



LINK

“SPA의 핵심은 새로고침 없는 화면 전환입니다.



페이지 로딩 방식의 변화

기존 웹사이트는 페이지 전체를 다시 로드하지만, SPA(Vue)는 주소만 변경하고 내부 컴포넌트만 교체하여 마치 페이지가 이동한 것처럼 보이게 합니다.



사용자 경험(UX) 향상

서버 렌더링 방식보다 훨씬 빠른 전환 속도를 제공하여, 사용자에게 앱과 같은 부드러운 경험을 제공합니다.

★ Vue Router 3대장

1

Routes

URL ↔ 컴포넌트 매핑 설정

2

RouterView

컴포넌트를 화면에 출력하는 영역

3

RouterLink

새로고침 없는 이동 링크 생성

"이 3가지만 알면 라우팅 기초 끝!"

Vue Router 핵심 개념 정리



Routes

CONFIGURATION

URL 경로와 컴포넌트를 연결하는 매핑 테이블입니다. 어떤 주소에 어떤 화면을 보여줄지 정의하는 설정의 중심입니다.



RouterView

OUTLET

현재 URL에 매칭된 컴포넌트가 렌더링되는 위치입니다. 주소가 바뀌면 이 영역의 내용만 교체됩니다.



RouterLink

NAVIGATION

HTML `<a>` 태그처럼 보이지만, 페이지 새로고침 없이 URL만 변경하여 부드러운 이동을 처리합니다.



SPA 라우팅의 이점

Why we use Vue Router?



빠른 화면 전환



부드러운 사용자 경험 (UX)



앱 같은 사용성



Vue Router 기본 디렉토리 구조

CLI로 생성한 기본 Vue 3 프로젝트에 라우터 설정을 위한 폴더와 파일들이 어떻게 구성되는지 확인해봅시다.



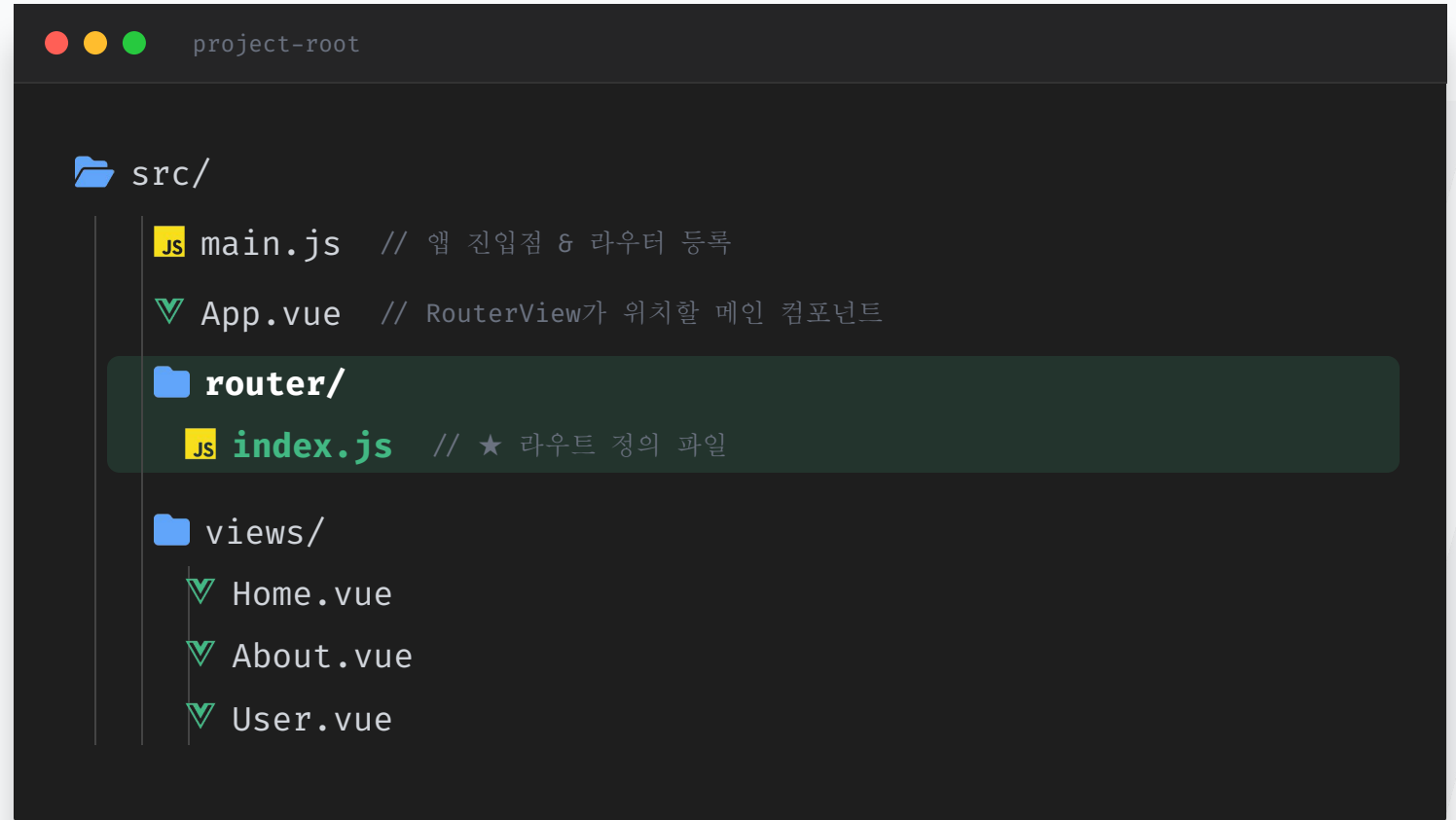
router/index.js

라우트 매핑 설정의 핵심 파일



views/

페이지 역할을 하는 컴포넌트들을 모아두는 폴더



```
npm install vue-router
```

Vue Router 설정의 핵심

URL과 컴포넌트를 매핑하고, 라우터 인스턴스를 생성하여 내보내는 전체 코드입니다.

1 라이브러리 & 컴포넌트 Import

vue-router 함수들과 각 페이지 컴포넌트를 가져옵니다.

2 Routes 배열 정의

path(URL)와 component(화면)를 객체 형태로 매핑합니다.

3 Router 인스턴스 생성

WebHistory 모드를 설정하고 routes를 주입합니다.

```
src/router/index.js

1 import {createRouter, createWebHistory} from 'vue-router'
2 import Home from '../views/Home.vue'
3 import About from '../views/About.vue'
4 import User from '../views/User.vue'
5
6 const routes = [
7   {path: '/', component: Home},
8   {path: '/about', component: About},
9   {path: '/user/:id', component: User}, // 동적 라우트
10 ]
11
12 const router = createRouter({
13   history: createWebHistory(),
14   routes,
15 })
16
17 export default router
```



JS main.js

앱 생성 및 라우터 플러그인 등록

Entry Point

```
src/main.js

import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index'

const app = createApp(App)
app.use(router) // 라우터 사용 등록
app.mount('#app')
```

Point:

app.use(router)를 호출해야 애플리케이션 전체에서 <RouterView>, <RouterLink> 등을 사용할 수 있습니다.

App.vue

네비게이션 및 라우터 뷰 배치

Root Component

```
src/App.vue

<template>
  <header>
    <RouterLink to="/">Home</RouterLink>
    <RouterLink to="/about">About</RouterLink>
  </header>
  <main>
    <!-- URL에 맞는 컴포넌트 출력 -->
    <RouterView />
  </main>
</template>
```

Point:

<RouterLink>는 새로고침 없는 a 태그 역할을 하며, <RouterView> 자리에 실제 페이지 컴포넌트가 렌더링됩니다.

2

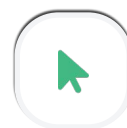
CHAPTER

라우트 등록 및 네비게이션 실습

라우트 정의와 RouterLink, 그리고 프로그래밍 방식 네비게이션 (router.push)을 실습합니다.



DEFINE



LINK



PUSH

이동은 간단하게, 제어는 강력하게



직관적인 라우트 등록

경로(path)와 컴포넌트(component)를 매핑하는 것만으로 등록이 완료됩니다. 복잡한 설정 없이 직관적인 구조를 가집니다.



자동 렌더링 시스템

RouterView는 현재 URL을 감시하고, 설정된 규칙에 따라 올바른 컴포넌트를 자동으로 화면에 갈아 끼워줍니다.

⇒ 페이지 이동의 2가지 방식



RouterLink

Template

HTML 태그처럼 사용하여 선언적으로 이동

```
<RouterLink to="/about">
```



router.push()

Script

로직 처리(로그인 등) 후 코드로 이동

```
router.push('/about')
```



스크립트에서 페이지 이동 제어하기

버튼 클릭이나 비동기 작업 완료 후 페이지를 이동시켜야 할 때는 router.push를 사용합니다.



useRouter Hook

Composition API에서는 useRouter를 통해 라우터 인스턴스에 접근합니다.



router.push()

히스토리 스택에 새 항목을 추가하여 뒤로 가기가 가능한 상태로 이동합니다.



팁: router.replace()를 사용하면 히스토리를 남기지 않고 현재 페이지를 교체합니다.

```
    <!-- 클릭 이벤트로 goAbout 함수 호출 -->
    <button @click="goAbout">About으로 가기</button>
  </div>
</template>

<script setup>
import { useRouter } from 'vue-router'

// 라우터 인스턴스 가져오기
const router = useRouter()

function goAbout() {
  // 프로그래밍 방식 네비게이션
  router.push('/about')
}
</script>
```

▼ About.vue

일반적인 컴포넌트 구조

View Component

views/About.vue

```
<template>
  <div>
    <h1>About 페이지</h1>
    <p>여기는 소개 페이지입니다.</p>
  </div>
</template>

<script setup>
  // 특별한 로직 없음
</script>
```

🔗 Template Navigation:

<RouterLink to="/about"> 태그를 사용하면 HTML에서 a 태그처럼 선언적으로 이동할 수 있습니다.



JS Programmatic

스크립트에서 이동 제어

Method Example

```
// 라우터 인스턴스 가져오기
const router = useRouter()

// 1. 히스토리에 추가 (일반 이동)
router.push('/about')

// 2. 히스토리 대체 (뒤로가기 불가)
router.replace('/about')

// 3. 뒤로 가기 (브라우저 Back)
router.back() // 또는 go(-1)
```

💡 Tip:

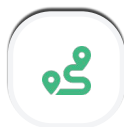
로그인 성공 후 리다이렉트처럼 로직 처리 후 페이지를 이동해야 할 때 router.push()를 주로 사용합니다.

CHAPTER

3

동적 라우팅과 파라미터 등록

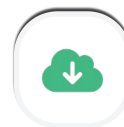
하나의 라우트 규칙으로 여러 페이지를 처리하는 동적 세그먼트 (:id)와 데이터 활용법을 배웁니다.



DYNAMIC



PARAMS



DATA

/user/1
/user/alice
/user/12345

라우터의 진짜 힘, 동적(Dynamic) 라우팅 파라미터 활용하기



URL 패턴 매칭의 효율성

사용자별 프로필처럼 구조는 같지만 데이터만 다른 수많은 페이지를 일일이 만들 필요 없이, 단 하나의 라우트 규칙으로 처리합니다.



데이터 기반의 화면 렌더링

URL에서 추출한 ID 값을 이용해 API를 호출하고, 해당 사용자에게 맞는 데이터를 동적으로 로딩하여 화면을 구성합니다.

⚡ Dynamic Matching

ROUTE DEFINITION

```
{ path: '/user/:id', component: User }
```



EXAMPLE URLS

/user/1

/user/alice

모두 같은 컴포넌트로 연결됨



ACCESS PARAMETER

```
const route = useRoute ()  
console.log (route.params.id )
```



파라미터 User.vue 구현

URL에 포함된 동적 ID 값을 컴포넌트 내부에서 읽어와 화면에 표시하는 방법을 알아봅니다.



useRoute()

현재 활성 라우트 정보에 접근하기 위한 Vue Router 훅(Hook)입니다.



route.params.id

URL에서 정의한 :id 부분의 실제 값을 가져옵니다.

```
User.vue

<template>
  <div>
    <h1>User 페이지</h1>
    <p>현재 유저 ID: <strong>{{ userId }}</strong></p>
  </div>
</template>

<script setup>
import { useRoute } from 'vue-router'

// 1. useRoute 훅을 사용하여 현재 라우트 객체 가져오기
const route = useRoute()

// 2. URL 파라미터(id) 읽기
const userId = route.params.id
</script>
```



JS router/index.js

동적 세그먼트(:id) 설정

Config

src/router/index.js

```
const routes = [  
  // ... 다른 라우트들  
  { path: '/', component: Home },  
  
  // 동적 세그먼트 :id 사용  
  { path: '/user/:id', component: User },  
]
```

URL 매칭 예시:

```
/user/1  
/user/alice  
/user/12345
```

Vue User.vue

파라미터 값 읽기

Script Setup

src/views/User.vue

```
import { useRoute } from 'vue-router'  
import { watch } from 'vue'  
  
const route = useRoute()  
  
// 현재 URL의 파라미터 값 읽기  
const userId = route.params.id
```

Tip: 반응성 처리

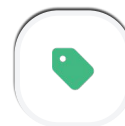
동일 컴포넌트 재사용 시 데이터 갱신을 위해 watch 사용:
watch(() => route.params.id, (newId) => { ... })

네비게이션 가드로 접근 제어하기

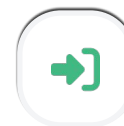
requiresAuth 메타 필드와 전역 가드를 사용하여 로그인되지 않은 사용자의 접근을 차단하고 리다이렉트합니다.



GUARD



META



REDIRECT

네비게이션 가드 개념과 시나리오

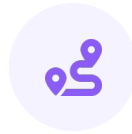
네비게이션 가드의 종류



전역 가드 (Global)

`router.beforeEach()`

애플리케이션의 모든 라우트 이동 시 실행됩니다.
로그인 체크 같은 공통 로직 처리에 가장 적합합니다.



개별 라우트 가드

`beforeEnter: (to, from)`

특정 라우트 설정(routes) 객체 내부에 정의합니다.
해당 경로로 진입할 때만 특별히 체크할 때 사용합니다.



컴포넌트 내부 가드

`onBeforeRouteLeave()`

컴포넌트 내부에 정의합니다. 작성 중인 폼이 있을 때 이탈
방지 경고 등을 띄울 때 유용합니다.

이번 챕터 실습 시나리오: 로그인 보호

1

로그인 페이지

/login 접속
버튼 클릭



2

인증 상태 저장

`localStorage.loggedIn`
`= 'true'`



3

보호된 페이지

/mypage 이동 시도
(`meta.requiresAuth`)



4

전역 가드 실행

조건 확인 후
허용 또는 리다이렉트



접근 제어 로직 구현하기

라우트에 메타 정보를 추가하고, 전역 가드에서 이를 확인하여 로그인 여부를 판단합니다.



Route Meta Fields

meta 속성에 사용자 정의 데이터(예: requiresAuth)를 추가하여 라우트별 설정을 정의합니다.



Global Guard

beforeEach는 모든 라우트 이동 전에 실행됩니다. 여기서 로그인 상태를 체크하고 next()로 제어합니다.

```
{
  path: '/mypage',
  component: MyPage,
  // ✨ 인증이 필요한 페이지 표시
  meta: { requiresAuth: true }
},
```

```
]
```

```
const router = createRouter({
  history: createWebHistory(),
  routes,
})
```

```
// 🛡️ 전역 네비게이션 가드 설정
```

```
router.beforeEach((to, from, next) => {
```

```
  // 로컬 스토리지에서 로그인 상태 확인 (가짜 로그인)
```

```
  const isLoggedIn = localStorage.getItem('loggedIn') === 'true'
```

```
  // 이동할 페이지(to)가 인증을 요구하고 && 로그인이 안 된 경우
```

```
  if (to.meta.requiresAuth && !isLoggedIn) {
```

```
    // 로그인 페이지로 리다이렉트
```

```
    return next('/login')
```

```
  }
```

```
  // 통과
```

```
  next()
```

```
})
```

<예시>

```
{
  path: '/admin',
  component: AdminPage,
  meta: {
    requiresAuth: true,
    role: 'admin'
  }
}
```

가짜 로그인 시나리오 구현

백엔드 서버 없이 클라이언트 측 저장소를 활용하여 인증 흐름을 흉내 냅니다.



localStorage 활용

'loggedIn' 키에 값을 저장하여 로그인 상태 유지



Router Push

로그인 성공 후 /mypage로 프로그래밍 방식 이동



비동기 흉내 (UX)

setTimeout으로 네트워크 지연 효과 연출

```
13 import { useRouter } from 'vue-router'
14
15 const router = useRouter()
16 const message = ref('')
17
18 function login() {
19   // 1. 로컬 스토리지에 로그인 상태 저장
20   localStorage.setItem('loggedIn', 'true')
21   message.value = '로그인 성공! 이동합니다...'
22
23   // 2. 잠시 후 마이페이지로 이동 (UX)
24   setTimeout(() => {
25     router.push('/mypage')
26   }, 500)
27 }
28 </script>
29
```

인증된 사용자 전용 My Page 컴포넌트

로그인한 사용자만 볼 수 있는 화면을 구성하고,
로그아웃 기능을 통해 인증 정보를 삭제하는 로직을
구현합니다.

➡ 로그아웃 로직

localStorage에서 loggedIn 키를 삭제하여
인증 상태를 초기화합니다.

↪ 리다이렉션

로그아웃 후 router.push('/login')을 사용하여
로그인 페이지로 사용자를 이동시킵니다.

```
src/views/MyPage.vue
13 <script setup>
14   import { useRouter } from 'vue-router'
15
16   const router = useRouter()
17
18   function logout() {
19     // 1. 인증 정보 삭제
20     localStorage.removeItem('loggedIn')
21
22     // 2. 로그인 페이지로 이동
23     router.push('/login')
24   }
25 </script>
26
```

정리 및 다음 단계



Mission Complete

✓ 오늘 배운 내용

Vue Router의 핵심 기능을 마스터했습니다.



SPA 라우터 구조 이해

Routes, RouterView, RouterLink의 역할



라우트 등록 및 이동

RouterLink와 router.push() 활용법



동적 라우팅 활용

/user/:id 파라미터 처리와 useRoute()



접근 제어 (Navigation Guard)

meta 필드와 전역 가드로 로그인 보호 구현

↑ 도전 과제

더 견고한 애플리케이션을 위해 학습해보세요.



데이터 손실 방지

beforeRouteLeave를 사용하여 작성 중인 폼에서 나갈 때 경고창 띄우기



권한(Role) 기반 라우팅

단순 로그인을 넘어 관리자(Admin)와 일반 유저를 구분하여 접근 제어하기



404 페이지 처리

존재하지 않는 경로로 접근 시 NotFound 컴포넌트 보여주기